Falls Game

Final Report for the Distributed Systems Course [A.Y. 2024/2025]

Guo Jiahao
jiahao.guo@studio.unibo.it

October 31, 2026

This report presents the "Falls Game" project, realized for the "Distributed Systems" course at the University of Bologna. The project is a multiplayer game inspired by one of the levels of *Fall Guys*: players compete in a multi-layered arena where the platforms disappear beneath their feet, and the last one standing wins. The game is written in Python with the Ursina engine. The main focus of the project is the networking. The matchmaking mode relies on a matchmaking server that creates a separate game process for each match, while in the peer-to-peer mode there is no server: all the players run the same program and one of them, the host, also acts as the authority of the match (a peer-to-peer architecture with a host). Both modes are played on a local network; hosting the matchmaking server on a cloud service is left as future work. In both modes an authoritative host or server decides the state of the match and the clients only send their positions and requests. The system uses TCP for the operations that must be reliable (matchmaking and joining) and UDP for the real-time game traffic, and it uses Redis to keep track of the matches in progress. It also tolerates failures: a player who loses the connection can rejoin the match, and a peer-to-peer match survives the loss of its host thanks to an automatic host migration.

Disclaimer

During the preparation of this work, the author used AI to write the project documentation and the final report. After using this tool, the author reviewed and edited the content as needed and takes full responsibility for the content of the final report.

1 Concept

1.1 Use case description

Falls Game is a multiplayer desktop game inspired by one of the levels of *Fall Guys*. It is written in Python with the Ursina engine. The players run on a multi-layered arena made of blocks that disappear shortly after they are stepped on. A player who falls off the arena is eliminated, and the last one standing wins.

Users are located on separate PCs of the same local network, with two or more players in a match (the number is set in `config.py`). Distribution is therefore a fundamental requirement: every player must see the same arena, the same blocks disappearing and the same winner, even if the game runs on different machines. There are two ways to play. In the *peer-to-peer (P2P)* mode there is no server machine: all the players run the same program, one of them hosts the match (it is a player and the authority at the same time) and the others find it on the network and connect to it. In the *matchmaking* mode a server collects the players in a queue and starts a game process for them.

Interaction is continuous during a match: every client sends the position of its player at every frame, and the host (or the game server) sends back the positions of all the players many times per second. Players use the keyboard and the mouse to play (`W`, `A`, `S`, `D` to move, the mouse to look around, `Space` to jump and the left mouse button to dash), and a menu to choose the mode. Users interact with the game whenever they launch it, typically for casual sessions on PC.

The system does not store any persistent data. The state of a match lives only in the memory of the process that runs it. Redis is used only as a short-lived registry of the matches that are running, and each client keeps a small local file that lets it rejoin a match after an interruption.

The system has the following roles:

- **Player**: joins a match, moves its own avatar and sends requests (positions, steps on blocks, falls) to the authority.
- **Host (P2P)**: a player who is also the network hub and the authority of the match. It is a role, not a special machine: any player can be the host, and if the host disappears another player takes over.
- **Matchmaking server**: collects the players in a queue, starts a game process for each match and tells the players where to connect.
- **Game server** (`game_instance`): one process per match, with the same authority role as the P2P host but without playing.

2 Requirements

2.1 Glossary

Term	Definition
Match	One game, from the lobby to the declaration of the winner.
Lobby	The phase before the start, where players are connected and wait for the others.
P2P mode	Peer-to-peer mode *with a host*: there is no server; all the players run the same program and one of them, the host, is also the authority and the hub of the match, on the local network.
Host	In P2P mode, the player that holds the authority of the match and relays the traffic. Any player can be the host.
Matchmaking mode	Mode where players enter a queue and a server creates the match.
Matchmaking server	The process that manages the queue and starts a game server for each match.
Game server	The `game_instance` process that runs one match in matchmaking mode.
Authority	The component that decides the rules of the match (`GameAuthority`): the P2P host or the game server.

Term	Definition
Block	A tile of the arena that disappears after a player steps on it.
Floor seed	A number sent at the start: every client uses it to build the same floor.
Snapshot	A message with the positions of all the players, sent many times per second.
Heartbeat	A periodic message that tells the other side that a component is still alive.
Silent player	A player from whom the host has received nothing for 5 seconds.
Grace period	The 10 seconds that a silent player has to come back before being removed.
Rejoin	A player who left by accident returns to the same match with the same identity.
Host migration	The election of a new P2P host when the old one disappears.

2.2 Functional requirements

Description	Acceptance criterion
FR1 In matchmaking mode, a player must be able to enter a queue and be put in a match with other players.	As many clients as `MAX_PLAYERS` enter the queue and receive the address of the same match and join it. If no match can be started, the client receives an explicit refusal.
FR2 In P2P mode, a player must be able to host a match on the local network and the others must be able to find and join it.	The host appears in the list of the other players within a few seconds. Each joining player receives an id. A player who joins a full room is refused.
FR3 The match must start for all players at the same time.	Once everybody has loaded the level, a 3-2-1 countdown is shown to all and the game starts. If somebody never reports ready, the countdown starts anyway after 10 seconds.
FR4 A player must be able to move, jump and dash.	Pressing the keys moves the avatar according to the physics, and the other players see it moving.
FR5 A block stepped on must disappear for all players, and only the first player who claims it counts.	A single `block_destroyed` message is sent to all, and the same blocks are missing on every screen.
FR6 A player who falls off the arena must be eliminated, and the last player standing must be declared the winner.	When a player falls, everybody is told that he is out; when one player is left, everybody receives the winner.
FR7 The positions of all the players must be visible in real time.	Snapshots are sent in fixed time, and the other avatars move without visible jumps.
FR8 A player who loses the connection or closes the window by mistake must be able to return to the match.	Within 10 seconds the player rejoins with the same identity, receives the current state and the match resumes for everybody. After 10 seconds he is removed and the others go on.
FR9 In P2P mode, the match must survive the loss of the host.	The player with the lowest id becomes the new host, the others reconnect to it and the state of the match is kept.

2.3 Non-functional requirements

Description	Acceptance criterion
NFR1 Responsiveness: the movement of a player must not wait for the network.	A client moves its own avatar immediately. Positions travel on UDP: a lost packet is replaced by the next one.
NFR2 Consistency: all the players must see the same state of the match.	The rules are applied in one place only (the authority), and the automatic tests check them (first claim wins, win condition, dead players ignored).

Description	Acceptance criterion
NFR3 Fault tolerance: the failure of a player must be detected and must not stop the others for ever.	A player silent for 5 seconds causes a pause for everybody, and after the 10 seconds of grace period he is removed. Invalid packets are dropped and never crash the host.
NFR4 Robustness of the matchmaking: a running match must survive a restart of the matchmaking server.	After the restart, the match goes on and the new server adopts it.
NFR5 Basic security: only the players chosen for a match can enter it, and nobody can act on behalf of another player.	An unknown address is refused (`UNAUTHORIZED`); a packet with a false `sender_id` is dropped.

3 Design

3.1 Architecture

The game supports two different architectures, one for each mode. Both share the same code for the rules and for the network; what changes is where the authority runs and how the players find the match.

- **Matchmaking mode.** A dedicated matchmaking server collects the players in a queue and starts a separate server process (`game_instance`) for each match. This process is the authority of the match: it applies the rules and sends the state to the clients. The players only play, and the server is not one of them.
- **P2P mode.** There is no server machine: the players are on the same local network, every participant runs the same program, and one of them *hosts* the match, acting both as a player and as the authority. The other players connect to the host and find it by scanning the network. It is a *peer-to-peer architecture with a host*, not a pure (full mesh) one: the roles are not fixed, because any participant can be the host and, if the host disappears, another player takes over (host migration); but at any moment the topology is a *star* centred on the host, because the clients never talk to each other and all the traffic goes through it.

The choice of a host, and not of a full mesh, is deliberate. In a full mesh every peer would apply the rules by itself, and two peers could decide in a different way who stepped first on a block, or who is the winner. With a single host there is a single place where a conflict is solved, and this is what keeps the state of the match consistent. The price is that the host is a single point of failure and a bottleneck: this is why the system has a host migration, and why it is meant for a few players on a local network.

	Matchmaking	**P2P with a host**
Who is the authority	the game server process	the host (a player)
Dedicated machine	yes, the server PC	no
Topology	star, centred on the server	star, centred on the host
Roles	fixed	any player can be the host
If the authority fails	the match ends	host migration

In both architectures the authority is a single component, so there is always a single place where the rules are applied.

The authority holds the rules of the match, and the network layer (`NetworkManager`) holds the communication: both are the same in the two variants, only the way the authority is started changes. Even the code that applies a request and tells everybody what happened (`apply_and_broadcast`) is the same: only the P2P host also shows each event on its own screen, because it is a player too. The clients only *ask* (for example "I step on this block") and the authority decides and tells everybody. This limits cheating and desynchronization, and gives a single place where a conflict between two players is solved.

The interaction is **event-driven**: a thread reads the network and puts the messages in a queue, and the game reads the queue at its own pace. The client is organized as a model (`GameModel`), views and a controller (`GameController`), close to the Model-View-Controller pattern, so that the network never touches the graphics directly.

The price of this choice is that the authority is a single point of failure. In matchmaking mode, the process of a match is isolated from the others and from the matchmaking server. In P2P mode a host migration lets the match go on.

3.2 Infrastructure

Infrastructural components

- **Clients:** one for each player, at most `MAX_PLAYERS` per match. A client sends its requests to the authority and receives the events.
- **Matchmaking server:** It keeps the queue of the players, starts a game process when `MAX_PLAYERS` players are waiting (at most `MAX_ACTIVE_GAMES` matches at the same time) and tells them to join a match. It also announces itself on the local network, so that the clients can find it.
- **Game server** (`game_instance`): one process per match, started by the matchmaking server as a detached process. It is the authority of that match.
- **P2P host:** one of the players, when the P2P mode is used. It is the authority and the network hub of the match.
- **Redis:** Used as a registry of the running matches.
- **Dashboard**: a read-only page that shows the queue and the matches published on Redis.

Network distribution

Everything is deployed on a local network. In matchmaking mode, the matchmaking server, Redis and all the game processes run on the same machine, and the clients are on other machines of the same network. In P2P mode there is only the machine of the host and those of the other players.

Service discovery

In *matchmaking mode*, the matchmaking server announces itself with a UDP broadcast on the local network once per second, on its own port (`15001`), with the same message as a P2P host but with the id `-1`: in this way the P2P lists do not show it. A client waits up to 2 seconds for this announcement, so no address has to be configured. If the broadcast does not reach the client (for example because of a router), the address of the server can be set by hand in the environment variable `MATCHMAKING_HOSTNAME`. The clients do not need to know where a match is: the server answers with `GAME:<host>:<port>`, and the clients then talk directly to the game process. In *P2P mode*, the host announces itself with a UDP broadcast on the local network once per second, from every network card of its PC (`GAME_HOST:<player_id>:<port>`), and the clients listen and show the hosts found; a host that announces itself from several cards is shown only once. After a host migration, the new host announces itself in the same way.

Redis is used only on the server side: the clients never talk to it.

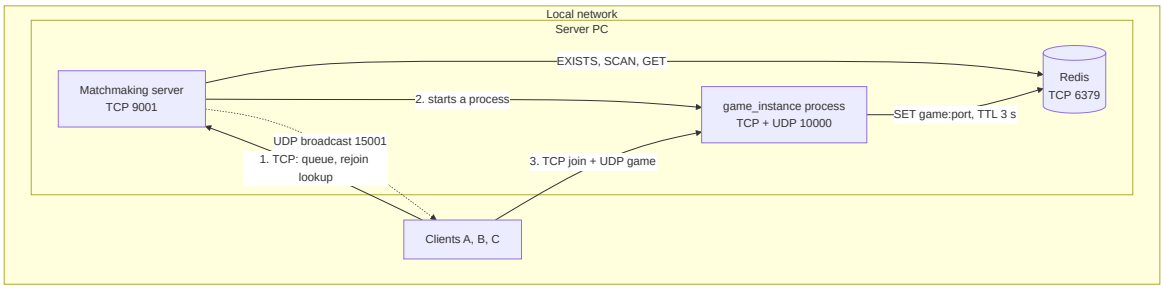


Figure 1: Matchmaking deployment

In P2P mode there is no server machine: one player is the host, which is at the same time the network hub, the game authority and a player. The other players are its clients. They never talk to each other, so the topology is a star centred on the host: all the traffic goes through it.

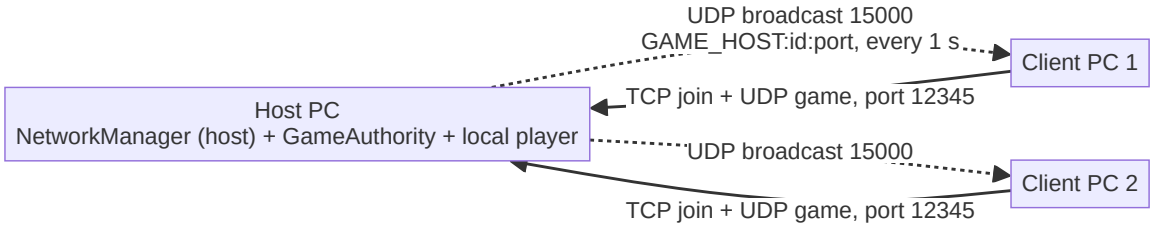


Figure 2: P2P with a host: a star centred on the host

3.3 Modelling

Domain entities and where they live

- **Match state**: the destroyed blocks, the positions of the players, who is alive and who has won, and the floor seed. It lives only in the memory of the authority, which is the single source of truth.
- **Room**: the roster of the network layer. It maps each player address to an id and records who is silent, who is rejoining, which players are expected and whether the match has started. It lives in the authority too.
- **Player avatar:** each client moves its own avatar and sends the position to the authority, which relays it to the others in the snapshots. The other players are replicas on the screen of each client.
- **Blocks and floor:** the authority decides which blocks are destroyed. Every client builds the same floor from the floor seed and removes the blocks that the authority announces.
- **Queue and running games:** the queue is in the memory of the matchmaking server. The list of the running games is on Redis, so that it survives a restart of the matchmaking server.
- **Rejoin data:** a small file on the client (`.rejoin/player_<id>.json`) with the identity needed to come back to a match.

State of the system

The state is divided in four layers: the *session*, the *match*, the *replica* on each client (positions, destroyed blocks, what is on the screen), and the *registry* of the running games.

Domain events and messages

The messages are of four kinds:

- **Requests** from a client to the authority: `update_pos`, `block_step`, `player_ready`, `player_died`, `rejoin_done`, `client_heartbeat`, and the control words `DISCONNECT` and `LEAVE`.
- **Events** from the authority to all: `spawn_player`, `block_destroyed`, `player_removed`, `game_won`, `game_pause`, `player_left`, `start_game`, `start_countdown`, `host_left`.
- **Continuous traffic:** `state_snapshot` and `host_heartbeat`.
- **Replies and queries** on TCP: the join handshake and the matchmaking tokens (`WAIT`, `GAME:...`, `SERVER_FULL`, `ID_COLLISION`, `NOT_FOUND`, `CANCEL`).

`GameController` drives the game, `GameModel` keeps its state, `NetworkManager` is the only class that touches the network, and `GameAuthority` holds the rules and the state of the match (on the host or the game server only).

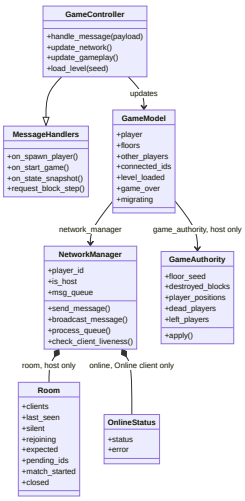


Figure 3: Main classes and who owns whom

3.4 Interaction

The interaction follows a player from the queue to the end of a match. Messages marked *TCP* are reliable and used once; messages marked *UDP* are the continuous game traffic and low latency.

Discovery and join

In matchmaking mode the client first finds the server (with its UDP announcement, or with `MATCHMAKING_HOSTNAME` if it is set), then opens a TCP connection to it. The server answers `WAIT` until `MAX_PLAYERS` players are in the queue and then `GAME:<host>:<port>`. The client then joins the game process with a TCP handshake that carries its own UDP port: from then on the server knows where to send the game traffic.

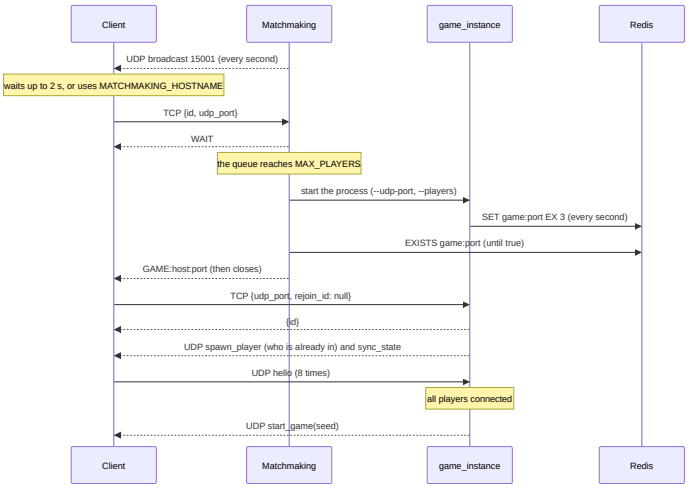


Figure 4: Matchmaking: from the queue to the match

In P2P mode the client discovers the hosts through the UDP broadcast and joins the chosen one with the same TCP handshake.

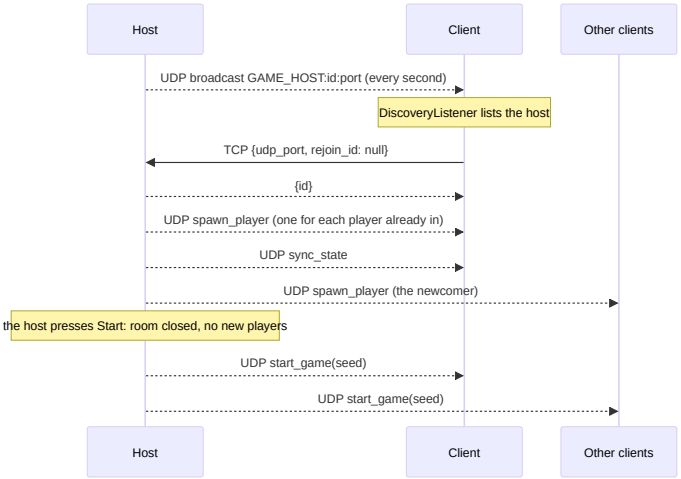


Figure 5: P2P: discovery, join and start

Start

Each client loads the level and, when it is ready, tells the authority. When everybody is ready the authority sends `start_countdown`.

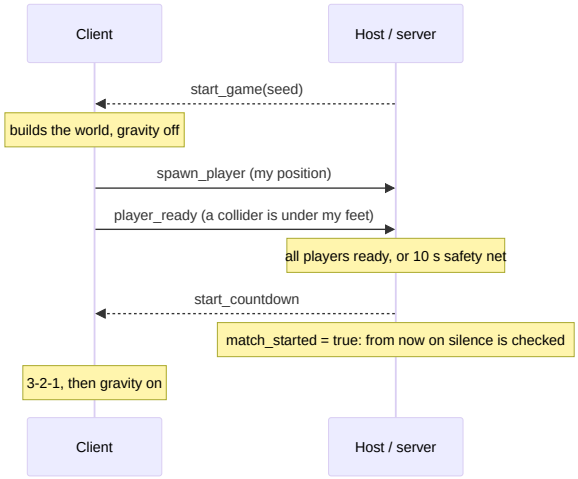


Figure 6: Loading, ready and countdown

During the match

The authority never trusts the clients: a client only *asks*, the authority decides and tells everybody. While playing, the clients send their position at every frame, and the authority sends the snapshots with all the positions to everybody. On the client, a snapshot does not move the other avatars at once: it only sets their target position, and at every frame each avatar moves a little towards it. In this way the movement looks smooth even if the snapshots arrive less often than the frames are drawn, while the screen is redrawn 60 times a second or more.

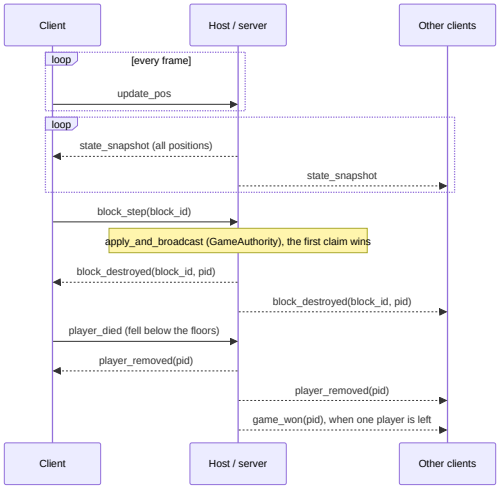


Figure 7: During the match

Failures

The host sends a heartbeat every second, and the clients answer. The same pause is triggered by 5 seconds of silence (for example a crash or a lost connection). The player can rejoin during the grace period, and if the host disappears in P2P mode, a new one is elected.

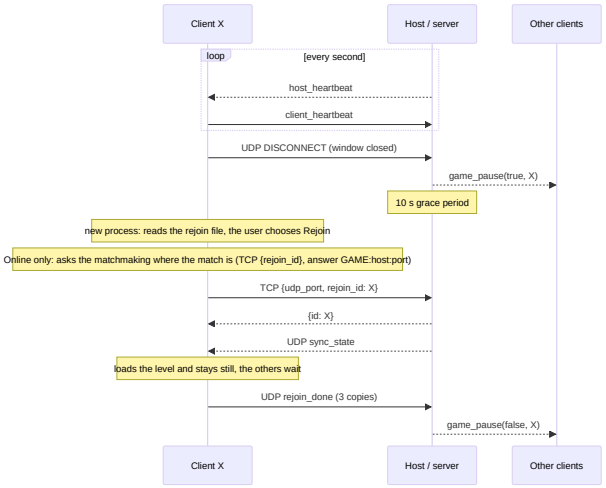


Figure 8: Heartbeat, disconnection and rejoin

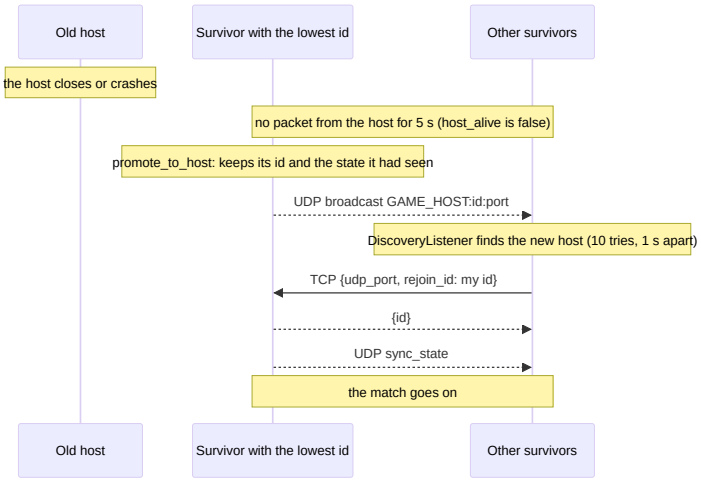


Figure 9: Host migration (P2P)

3.5 Behaviour

The components that decide are *stateful* (the authority, the matchmaking server and Redis); the clients keep a replica of the state that the authority sends them. The matchmaking server can lose its memory without losing the matches, because it rebuilds the list of the running games from Redis.

Client in matchmaking mode

Before the match, a client goes through the states of the figure: it connects to the matchmaking server, waits in the queue, and when it receives the address of the match it joins it. An error (server not reachable, `SERVER_FULL`, `ID_COLLISION` or a lost connection) ends the search with a message.

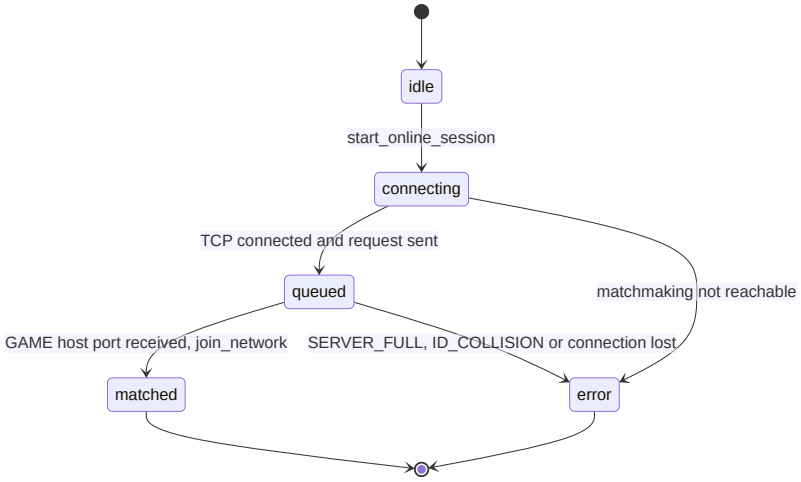


Figure 10: State of a matchmaking client (`OnlineStatus.status`)

Player, as seen by the host

The host is the only component that decides when a player is paused, rejoined or removed. A player is *active* while his packets arrive. If nothing arrives for 5 seconds, or if he closes the window during a match, he becomes *silent* and the match is paused for everybody. If he comes back within 10 seconds, or any packet arrives, he is active again, otherwise he is removed.

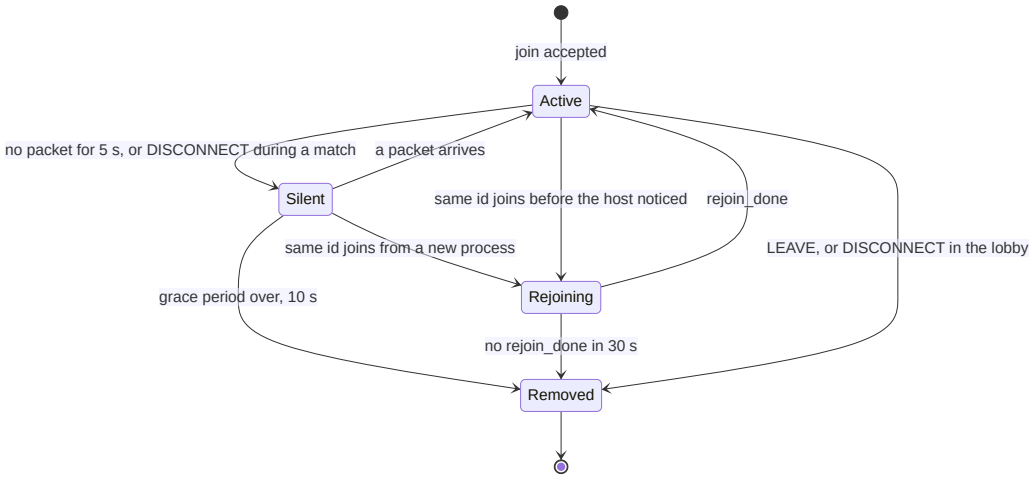


Figure 11: State of a player, as the host sees him

Game server

The game server waits for the players, sends `start_game`, waits for everybody to be ready and then runs the match. It closes itself when nobody has been connected for 3 seconds, and it removes its key from Redis.

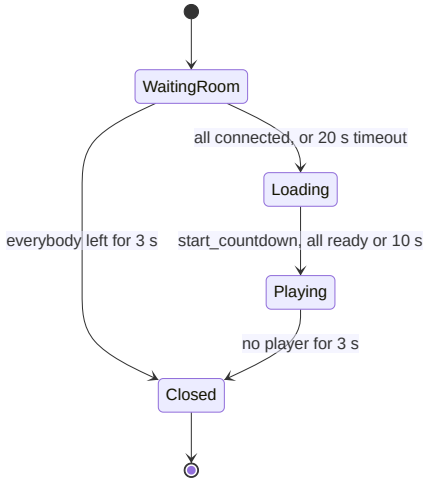


Figure 12: Life of a match

Client during the game

The controller of the client follows the phases of the game: menu, lobby, loading, countdown and play. During the play the game can be paused and resumed by the authority.

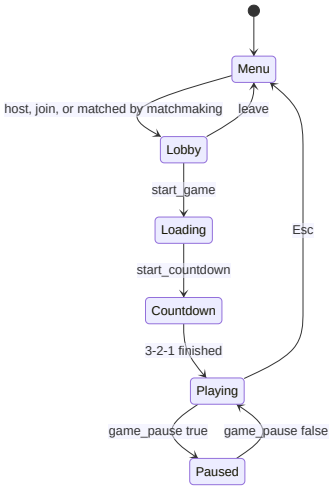


Figure 13: What the player sees, on the client

3.6 Data and Consistency Issues

- **What is stored.** No data survives a match, so there is no database for the game. Two small pieces of data are kept on purpose: the registry of the running games on Redis (the key `game:<port>` of each match, and `matchmaking:status` with the queue for the dashboard) and, on the client, the rejoin file.
- **How it is stored.** Redis is a key-value store: the data is a few keys with a small JSON value, no relations are needed, and the *expiration time* of a key (TTL) gives exactly what is needed: if a game process crashes, its key disappears by itself.
- **Who queries Redis.** Each game process writes its own key every second with a TTL of 3 seconds. The matchmaking server reads the keys when it starts, to adopt the matches that are

still running, and checks that an adopted game is still alive. The dashboard only reads. A key is written by one process only, so there are no concurrent writes on the same key.
- **Shared data.** The state of the match is not shared between components: it is owned by the authority, and the clients receive a replica through the snapshots and the events. The only data given by the matchmaking server to a game process is the list of the players, passed when the process is started.

3.7 Fault-Tolerance

- **Replication.** The state of the match is not replicated for safety, but each client keeps a replica of what it needs to draw the match. A player who joins or rejoins receives the current state in a `sync_state` message. In P2P mode, the new host after a migration rebuilds the match from what it knew as a client (floor seed, destroyed blocks, positions), and the others receive it again when they reconnect.
- **Heartbeat, timeout and retry.** The host sends `host_heartbeat` every second and every client answers `client_heartbeat`, so that the host notices a silent client (5 seconds) and the clients notice a silent host. A silent client has a grace period of 10 seconds, and a player who rejoined has 30 seconds to finish loading the level. The game processes renew their Redis key.
- **Error handling.** A UDP packet that is not valid JSON, comes from an unknown address or carries a false `sender_id` is dropped and never stops the host. A refused join gets an explicit reason (`ROOM_FULL`, `REJOIN_EXPIRED`, `MATCH_STARTED`, `UNAUTHORIZED`). If Redis is not reachable when the server starts, the server stops and prints why; if Redis goes down later, the processes keep working and retry. If a game process cannot be started, the players receive `SERVER_FULL` and the port is freed. A match survives the restart of the matchmaking server, because each match is an independent process.

3.8 Availability

- **Caching.** There is no cache. The Redis keys with a TTL work as a short-lived registry, not as a cache of data.
- **Load balancing.** There is none. The matchmaking server starts each match on its own port (`10000 + n`) on the same machine.
- **Network partitions.** The CAP theorem says that, when the network is split, a distributed system must choose between staying consistent and staying available. A network can always fail, so the system has to tolerate partitions. When this happens, the game prefers to stay *consistent* rather than *available*. All the rules of the match are decided by a single authority, so every player sees the same state. If a player becomes unreachable, the match is paused for everybody instead of letting each side go on with a different state. The system reacts as follows:
  - **A client disappears.** The host sends a heartbeat every second (`host_heartbeat`) and records when it last heard from each client. A client silent for more than 5 s (`HEARTBEAT_TIMEOUT`) is marked as silent and the match is paused for everybody (`game_pause`).
  - **Grace period.** The client has 10 s (`REJOIN_TIMER`) to come back. After that it is removed and the others continue, so the pause is never infinite: availability is given up only for a limited time.
  - **Rejoin.** A returning client asks to join again with its saved id. The host replaces the old address with the new one and sends the current state (`sync_state`); the others wait until it has loaded the level.
  - **The host disappears.** Every client checks the host heartbeat; after 5 s without it, the player with the lowest id becomes the new host and the others rejoin it. A client looks for the new host once a second for 10 attempts and then gives up with an error message; a player who is left alone wins. In matchmaking there is no host migration: if the game process stops, the match ends.

– **Lost packets.** Positions are sent over UDP many times per second; a lost packet is not resent, because the next one replaces it. Only for this data the system accepts eventual consistency.

3.9 Security

- **Authentication.** There are no accounts. A client chooses a random label when it enters the queue, and the matchmaking server refuses duplicates. The system is meant for a local network and stores no sensitive data.
- **Authorization.** The authority decides what a player can do. Only the players chosen by the matchmaking server (their address and port) can join a match, and the host checks that the `sender_id` of every packet matches the address it comes from, so nobody can act on behalf of another player. The first claim on a block wins, and dead or departed players are ignored.
- **Cryptography.** None: the traffic is in clear text and there is no TLS or encryption. This is acceptable on a trusted local network, but it would have to change if the game was opened to the Internet.

4 Implementation

4.1 Network protocols

Three transports are used, each for what it does best.

Purpose	Protocol	Port	Why this protocol
Matchmaking: queue, `WAIT`/`GAME:` answers, rejoin lookup	TCP	`9001` (`MATCHMAKING_TCP_PORT`)	a request must never get lost, and a closed connection tells the server that the player left the queue
Join handshake with a game host (P2P) or game server (matchmaking)	TCP	same number as the game UDP port	joining must be reliable: the answer is either an id or an explicit refusal
In-game messages (positions, spawns, pause, ...)	UDP	P2P host `12345` (`P2P_PORT`), matchmaking game `10000 + n` (`GAME_INSTANCE_BASE_PORT`)	low latency; a lost position packet is replaced by the next one
LAN discovery of P2P hosts	UDP broadcast	`15000`	the host announces `GAME_HOST:<player_id>:<port>` every second, from every network card; clients listen and show the hosts found
LAN discovery of the matchmaking server	UDP broadcast	`15001` (`MATCHMAKING_DISCOVERY_PORT`)	the server announces itself every second; a client waits up to 2 s for it, so no address has to be configured
Game list shared by the server processes	TCP (Redis protocol)	`6379`	see *Databases* below

The host/game server opens **one UDP and one TCP socket on the same port number**: the client sends its own UDP port through the TCP handshake, and from then on the host knows where to send UDP packets.

4.2 Data representation

All data is text, so it can be read in a log.

- **In-game messages** (one UDP datagram each): JSON `{"method": "...", "args": [...], "sender_id": N}`. Requests of a client: `update_pos`, `block_step`, `player_ready`, `player_died`, `player_left`, `rejoin_done`, `client_heartbeat`. Events sent by the host/server: `spawn_player`, `state_snapshot`, `block_destroyed`, `player_removed`, `game_won`, `game_pause`, `start_game`, `start_countdown`, `sync_state`, `host_heartbeat`, `host_left`. Two control words are plain text, not JSON: `DISCONNECT` (window closed, the player may come back) and `LEAVE` (deliberate exit).
- **Join handshake** (TCP): request `{"udp_port": P, "rejoin_id": N or null}`; answer `{"id": N}` or `{"error": "<code>"}` with code `ROOM_FULL`, `REJOIN_EXPIRED`, `MATCH_STARTED` or `UNAUTHORIZED`.
- **Matchmaking** (TCP): the client sends `{"id": <label>, "udp_port": P}` to enter the queue, or `{"rejoin_id": N}` to ask where its match is. The server answers with a text token: `WAIT`, `GAME:<host>:<port>`, `SERVER_FULL`, `ID_COLLISION`, `NOT_FOUND`. A queued client can send `CANCEL`.
- **Redis values**: Key-Value. **Rejoin file** on the client: JSON as well.

4.3 Databases and how they are queried

Redis is the only database. It is a key-value store, queried through the `redis-py` client.

Key	Value	Written by	Read by
`game:<port>`	`{"port", "players", "pid", "start_ts"}`	the `game_instance` of that match, every second, with `SET ... EX 3`	the matchmaking server: `SCAN game:*` + `GET` at start-up, `EXISTS` to see if an adopted game is still alive
`matchmaking:status`	queue and active games	matchmaking server	the read-only dashboard

Why the TTL of 3 seconds: if a game process crashes nobody renews its key, so the game disappears from Redis by itself. When the matchmaking server is restarted it scans `game:*` and **adopts** the matches that are still running.

4.4 Authentication and authorization

There are no user accounts. Components are identified and checked in these ways:

- **Identity of a player**: the client invents a random label when it enters the queue.
- **Who can enter a match**: the matchmaking server tells the game server the exact list `(ip, udp_port) -> player id` of the players it has put together. Any other address gets `UNAUTHORIZED`.
- **Anti-spoofing**: for every UDP packet the host checks that the `sender_id` written in it is the id registered for the **source address**. Packets from unknown addresses, with a wrong id are dropped.
- **Authorization**: the server is *authoritative*. A client only asks (`block_step`, `update_pos`, ...), and `GameAuthority` decides: the first claim on a block wins, dead or departed players are ignored, malformed arguments are discarded, and the player who counts is the sender registered by the network layer, not the id written in the arguments. The id in the arguments is trusted only for *internal* messages, those born on the server itself (for example the disconnection of a client).

4.5 Technological details

- **Language and libraries**: Python 3, the **Ursina** engine for rendering and input, `redis-py` for Redis. Dependencies are managed with **Poetry**. Networking uses only the standard library: `socket`, `threading`, `json`, `subprocess`.

- **Threads**: every program has a main thread and some helper threads (receive, handshake, heartbeat, snapshot, accept, announcement). The game loop is never blocked waiting for the network. The complete list is in the subsection *Threads*.
- **One process per match**: the matchmaking server starts `game_instance.py` with `subprocess.Popen` as a *detached* process, with its log in `run/game_<port>.log`. A match survives a restart of the matchmaking server.
- **Timing constants**:

Constant	Value	Meaning
`HEARTBEAT_INTERVAL`	1 s	the host sends `host_heartbeat`, the clients answer `client_heartbeat`
`HEARTBEAT_TIMEOUT`	5 s	silence after which a client (or the host) is considered gone: the match is paused
`REJOIN_TIMER`	10 s	*grace period*: a paused player can come back within this time, otherwise he is removed
`REJOIN_LOAD_TIMEOUT`	30 s (`network.py`)	a player who rejoined must finish loading the level within this time
`JOIN_TIMEOUT_SECONDS`	20 s (`game_instance.py`)	the game server waits for the players, then starts with who is there
snapshot interval	`game_instance.py` (server), `network_p2p.py` (host)	how often the authority sends the positions of all players
`DISCOVERY_WAIT`	2 s (`network_online.py`)	how long a client waits for the announcement of the matchmaking server
`MAX_PLAYERS` / `MAX_ACTIVE_GAMES`	2 / 5	players per match / simultaneous matches; both can be changed in `config.py`

- **Rejoin after an interruption**: during a match every client rewrites a small file `.rejoin/player_<id>.json` once a second. At the next start the game offers to rejoin if that file is at least 2 s and at most `REJOIN_TIMER + 5` s old. In P2P mode the client joins the first host found on the LAN, and if that host refuses it the rejoin ends with the reason; in every case the search stops after 20 seconds.
- **Host migration**: if the host stops answering, the surviving player with the **lowest id** becomes the new host and announces itself on the LAN; the others find it, reconnect with the same id and receive the state.

4.6 Code structure and message flow

Every file also reads its constants from `config.py`, not drawn to keep the graph readable. Only `network.py` opens sockets for the game; the controller never uses a socket directly. The controller itself is split, in the same file, into classes for each task (sessions, start of the match, game loop, host migration, rejoin).

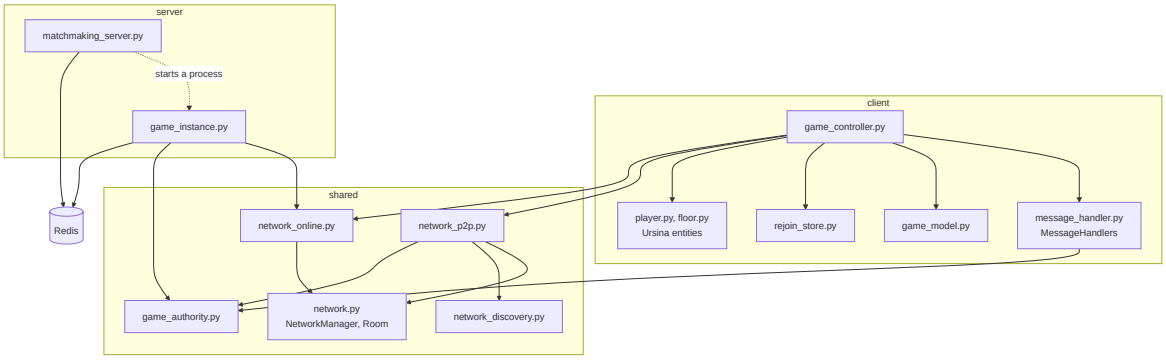


Figure 14: Relation between the files

Messages and what triggers them

Message	Sent by	Trigger	Handled by / effect
`update_pos`	client	every frame while playing (`update_gameplay`)	`GameAuthority.apply` stores the position; positions go out in `state_snapshot`
`block_step`	client	the player steps on a block (`request_block_step`)	`apply`: the first claim wins, then `block_destroyed` is broadcast
`player_died`	client	the player falls below a certain y (`request_player_died`)	`apply`: `player_removed`, and `game_won` if one player is left
`player_ready`	client	level loaded and a collider under the feet (`_wait_for_ground_then_ready`)	the host / server counts the ready players, then sends `start_countdown`
`spawn_player`	client; host	the client when its level is loaded; the host when somebody joins	the host relays the client's one to the others (`relay_broadcast`); clients create the remote player
`rejoin_done`	client	after rejoining, when the level is loaded (3 copies)	the host stops waiting for him and resumes the match
`client_heartbeat`	client	answer to every `host_heartbeat`	the host refreshes `last_seen`
`DISCONNECT`, `LEAVE`	client	`stop()`: window closed / deliberate exit	`handle_internal_messages`: pause with grace period / immediate removal
`host_heartbeat`	host / server	every 1 s	clients answer and refresh `last_host_seen`
`state_snapshot`	host / server	at the snapshot interval, tick thread	`on_state_snapshot` moves the other players
`block_destroyed`, `player_removed`, `game_won`	host / server	events returned by `GameAuthority.apply` and sent by `apply_and_broadcast`	clients update the scene
`game_pause(is_paused, pid)`	host / server	silence, `DISCONNECT`, rejoin in progress, removal	everybody pauses or resumes
`player_left(pid)`	host / server	`LEAVE`, `DISCONNECT` in the lobby, grace period over	clients remove the player
`sync_state`	host / server	`room.on_join` calls `send_state_to`: a player joins or rejoins	the newcomer receives destroyed blocks, positions and floor seed
`start_game`	host / server	P2P: the host presses Start; matchmaking: all players are connected	clients load the level

Message	Sent by	Trigger	Handled by / effect
`start_countdown`	host / server	all players ready	clients show 3-2-1 and then start
`host_left`	host	the host leaves on purpose (`_leave_network`)	clients know the host left deliberately

4.7 Threads

The system is made of four programs, and each one has its own threads. The rule is always the same: the threads that read the network (`receive_loop`, the handshakes) only put the messages in `msg_queue`, and only the *main thread* reads the queue and applies the rules with `GameAuthority`.



Figure 15: Programs, connections and threads

5 Validation

5.1 Automatic Testing

All tests use only Python's `unittest` and `unittest.mock`.

How to run them (from the project root):

```
poetry run python -m unittest discover -s tests -v          # All files
poetry run python -m unittest tests.test_network -v         # a single file
```

Overview

File	Level	What it checks
`test_models.py`	unit	`Room`, `OnlineStatus`, and all the rules of `GameAuthority`, including `apply_and_broadcast`
`test_network.py`	integration	roster, broadcast, room limits, leave/disconnect, silence and pause, anti-spoofing, join reliability, closed room, expected players
`test_rejoin.py`	integration	rejoin while paused or before the server noticed, refusal after the grace period, loading timeout, repeated crashes, matchmaking rejoin with a new UDP port
`test_host_migration.py`	integration	promotion of the lowest id, same id and state after reconnecting, strangers refused, the new host crashing too
`test_matchmaking.py`	integration	queue, `WAIT`/`GAME:`, duplicate ids, cancel, `SERVER_FULL`, rejoin lookup, adoption after restart, `game_instance` rules
`test_game_model.py`	unit	state of the client model: "in a match" or not, no shared state between instances
`test_rejoin_store.py`	unit	save/load/clear of the rejoin file, too fresh or too old saves

Rationale and techniques

- *Unit tests* cover pure rules without any I/O: the game rules and the data of a room.
- *Integration tests* start a real host and real clients on `127.0.0.1` in the same process, and send real packets. This tests the *communication* between components, which is where most bugs of a multiplayer game are.
- *Matchmaking tests* replace Redis with a small in-memory fake and the sockets with `socket.socketpair()`, so they are fast and do not need a server.
- *Time*: the real constants (5 s, 10 s, 30 s) would make the suite last minutes, so tests pass much shorter values to `check_client_liveness(timeout, grace_period)` and wait for events with a `wait_for(condition, timeout)` helper instead of fixed sleeps.
- *Isolation*: every file uses its own range of ports and `addCleanup` closes sockets and threads, so tests can run in any order.

5.2 Acceptance test

Manual tests were done on the full game, because they involve graphics, input, several machines or the network, which the automatic tests cannot reproduce.

#	Scenario	Expected result
1	LAN: one player hosts, two join from the scan list, host presses Start	3-2-1 countdown, everybody plays, last player alive wins
2	Matchmaking: `MAX_PLAYERS` clients (on as many PCs of the same local network, with no server address configured) enter the queue	all receive `GAME:...` and meet in the same match
3	A player closes the window and reopens the game within 10 s	the offer "do you want to rejoin?" appears; after loading, everybody resumes together
4	A player closes the window and does not come back	after 10 s he is removed; the others go on
5	The P2P host is closed during a match	the lowest id becomes host, the others reconnect, the floor state is kept
6	The matchmaking server is restarted while a match is running	the match goes on; the new server adopts it
7	Redis is not running when the server starts	the server stops with a clear message
8	A second group of players asks for a match while `MAX_ACTIVE_GAMES = 1` is full	the second group gets "server full"
9	Blocks disappear under the player's feet, jump, dash, special blocks	the same on every client

6 Release

The project is a **single repository** made of three folders, not a set of separately distributed packages:

- `shared/` – code used by both sides: `network.py`, `game_authority.py`, `network_online.py`, `network_p2p.py`, `network_discovery.py`, `config.py`;
- `server/` – `matchmaking_server.py` and `game_instance.py` and Dashboard.py;
- `client/` – Ursina game, views, controller, models, `rejoin_store.py`.

7 Deployment

Python 3 and **Poetry** are needed on every PC. The server PC also needs **Docker**, which is used to run **Redis**, but only for the matchmaking mode.

7.1 Install

Clone the Git repository, open a terminal in the project folder and install the dependencies:

```
poetry install --no-root
```

7.2 Install Redis with Docker

Redis runs on the PC of the matchmaking server; clients and P2P players do not need it. Install Docker, then start Redis in a container:

```
docker run -d --name redis -p 6379:6379 redis
```

Check that it answers (the reply must be `PONG`):

```
docker exec redis redis-cli ping
```

The next times the container already exists: `docker start redis`.

7.3 Play on P2P

No server and no Redis are needed. Every player is on the same network and starts the game:

```
poetry run python main.py
```

One player chooses *Host*, the others choose *Join* and pick the host from the list.

7.4 Play with the matchmaking server

On the server PC, in this order:

1. Start Redis:

   ```
   docker start redis
   ```

2. Start the matchmaking server:

   ```
   poetry run python server/matchmaking_server.py
   ```

3. Start the dashboard that shows the queue and the matches:

   ```
   poetry run streamlit run streamlit_dashboard.py
   ```

8 User Guide

8.1 Start the game

- *Matchmaking*: the game connects to the server and waits in the queue. When the queue is full (`MAX_PLAYERS` in `config.py`, 2 by default), the match starts. *Back* cancels the search.
- *Local* → *Host*: you create the match on your PC and the other players on the same network can find it.
- *Local* → *Join*: the game scans the network and lists the hosts; you choose one, or type its address and port.

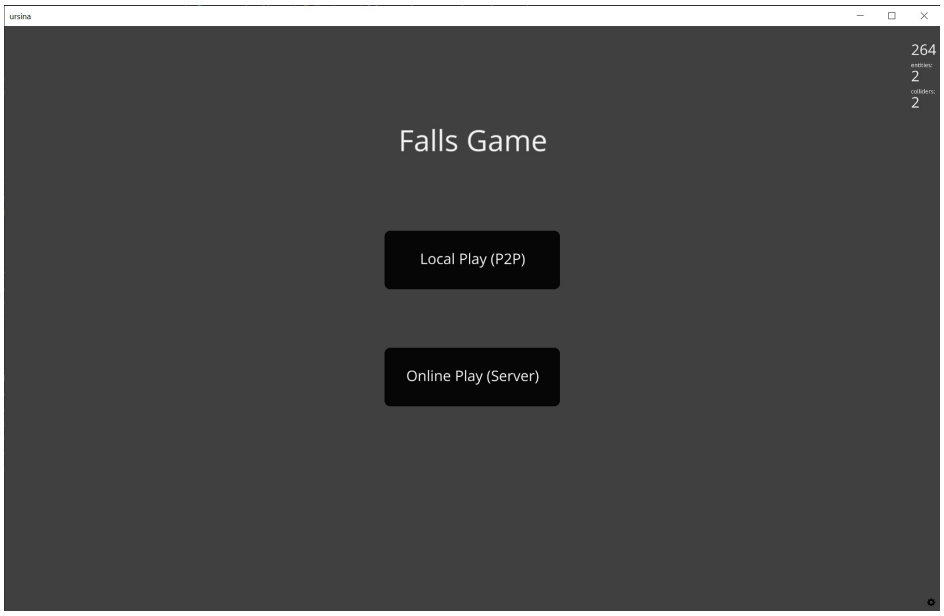


Figure 16: Choice of the game mode

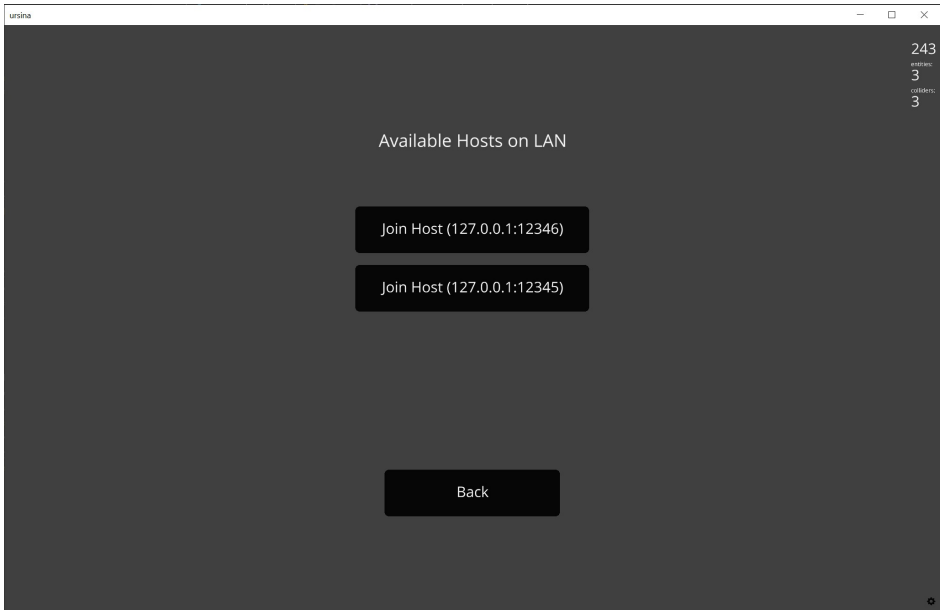


Figure 17: List of the hosts found on the network

8.2 Lobby and start

In a local match the host presses *Start* when all the players are in; in a matchmaking match the start is automatic. Everybody loads the level and waits for the 3-2-1 countdown, then the game begins.

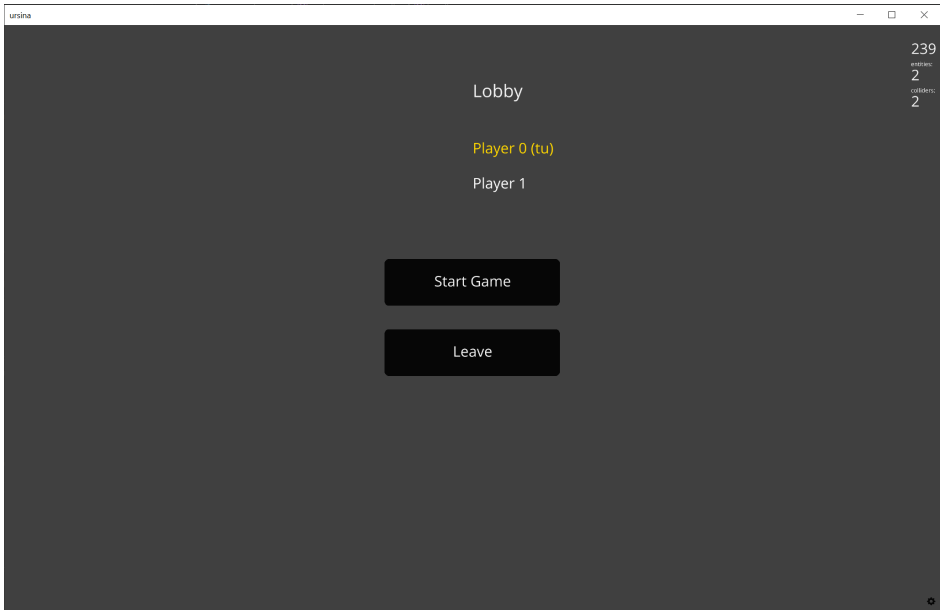


Figure 18: Lobby

8.3 Play

Move with `W`, `A`, `S`, `D`, look around with the mouse, jump with the `Space` bar and dash with the left mouse button. The blocks you step on disappear after a moment. If you fall off the floors you are out,

and the last player standing wins.

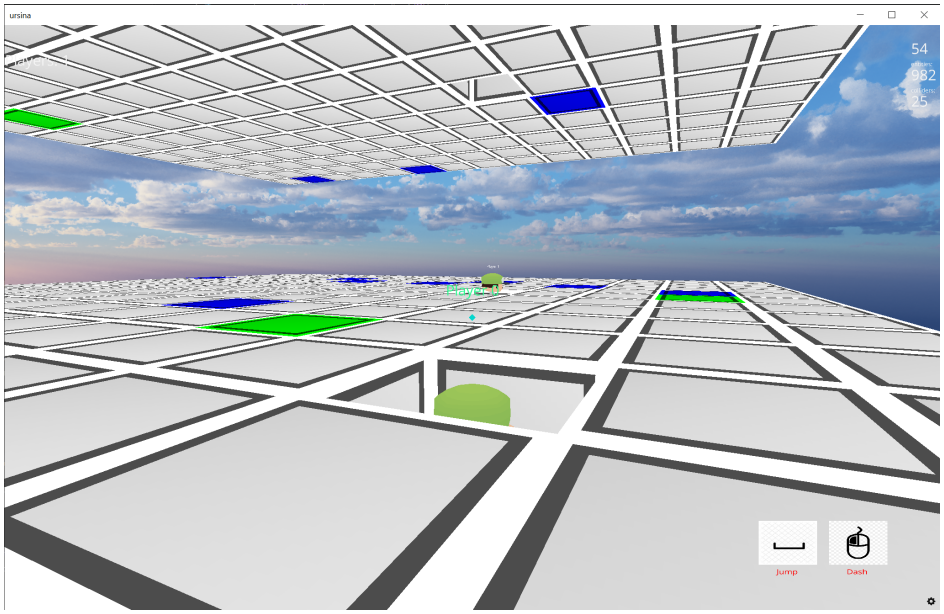


Figure 19: A match in progress

On the PC of the matchmaking server, the dashboard shows the players in the queue and the matches in progress, and it updates itself every 2 seconds.

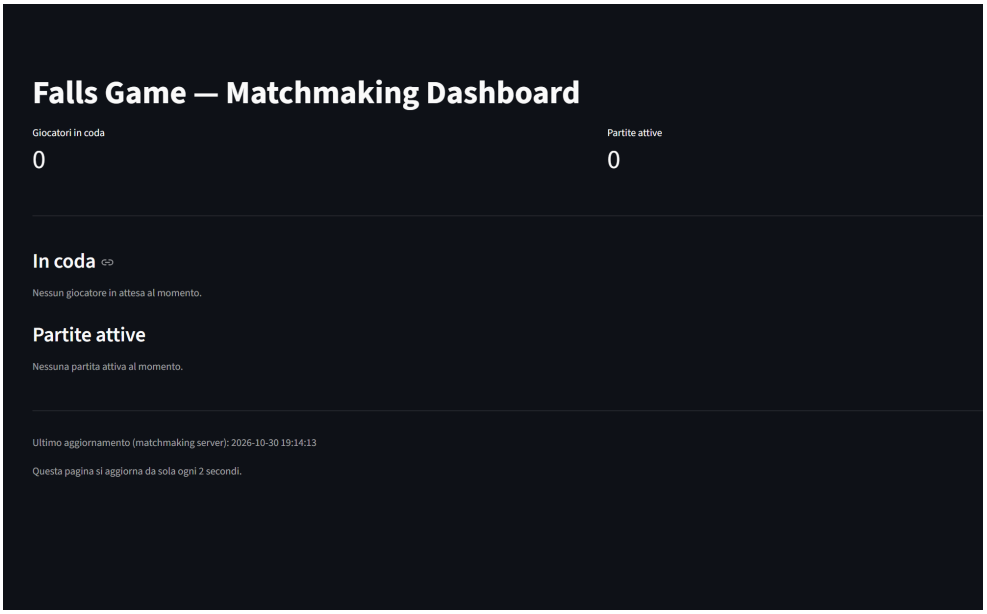


Figure 20: Dashboard of the matchmaking server

8.4 If something goes wrong

- If a player loses the connection, the match is paused for everybody for up to 10 seconds. If he comes back it goes on, otherwise he is removed.

- If you close the window by mistake, start the game again within a few seconds and choose *Rejoin*: you come back with the same identity and the match resumes for everyone.
- If the host of a local match disappears, the message "Host lost: election in progress..." appears and another player takes over.
- `Esc` during a match leaves it and goes back to the menu. This is a deliberate exit: you cannot rejoin.

9 Future works

The game already works as a distributed, real-time multiplayer game on a local network. These are the improvements that would make it more complete.

- **Playing over the Internet.** Today the game is tested only on a local network. The matchmaking server could be hosted on a cloud service, so that players on different networks can play together.
- **More security.** The traffic is in clear text and a player is only a random label. The connections could be encrypted and accounts could be added. The server could also check the positions sent by the clients (for example the speed), instead of trusting them.
- **A more robust game server.** If the process of a match stops, the match ends. The state of the match could be saved regularly so that a new process can continue the match, as the host migration already does in P2P mode.
- **Accounts and statistics.** With a database, the system could keep user profiles, wins and a leaderboard.